

Claude Code Skill 开发完全指南：从入门到精通

mp.weixin.qq.com/s/cn7mP_-M9wtfYK6RTOaajw

翔宇 workflow



Original

过去三个月，我用 Claude Code 创建了 30+ 个工作流 Skill：公众号自动发布、文章批量翻译、PPT 一键生成、代码审查……每个 Skill 都是一套完整的自动化流水线。

在这个过程中，我踩过无数坑：上下文爆满导致任务中断、步骤之间数据传递混乱、复杂流程无法断点恢复。反复试错后，我把这些经验沉淀成一套系统方法论——**Skill 规范**。

这套规范解决三个核心问题：

1. **如何设计**——Skill 的目录结构、文件组织、命名规则
2. **如何执行**——脚本与 Agent 的分工、上下文管理、断点恢复
3. **如何复用**——让任何人都能快速理解、修改、扩展你的 Skill

我把这套方法论沉淀成 7.8 万字的完整规范文档，并浓缩成这篇指南。掌握后，创建一个新 Skill 的时间从「摸索半天」缩短到「十分钟搞定」。

读完这篇文章，你会获得：

1. 一套完整的 **Skill 架构设计思路**——知道 Skill 长什么样，每个部分干什么
2. 一种「**分层递进**」的 workflow 思维——复杂任务如何拆解成可执行的步骤
3. 一份可直接复刻的 **Skill 开发模板**——文末提供完整提示词，从零搭建你的第一个 Skill

前置知识要求：了解 Claude Code 基本操作即可，不需要编程基础。

什么是 Skill？为什么需要规范？

Skill 的本质：给 AI 的「标准作业流程」

用一句话解释：**Skill 就是一个文件夹，里面装着告诉 Claude 「做什么、怎么做、做成什么样」的说明文件。**

打个比方。你去星巴克点一杯拿铁，店员不需要你解释什么是拿铁、用什么豆子、加多少奶。因为星巴克有标准作业流程（SOP），每个店员只要按 SOP 做，出品就是稳定的。

Skill 就是 Claude 的 SOP。

没有 Skill 的情况：你每次都要从头告诉 Claude 「帮我写一篇公众号长文，风格要像某某博主，结构是先抛问题再给方案，最后要有金句……」——费时费力，而且每次说法不一样，效果也不稳定。

有了 Skill 的情况：你只需要说「写公众号长文」，Claude 自动加载对应的 Skill，按照你预设好的风格、结构、流程来执行，输出质量稳定一致。

说人话

Skill = 可复用的 AI 工作模板。写一次，用无数次。

为什么需要「规范」？

你可能会问：我直接写个 SKILL.md 文件不就行了，为什么还要搞这么多规范？

原因有三个：

第一，Claude 的「记忆」是有限的。Claude Code 的上下文窗口是 200k token，听起来很大，但一个复杂工作流跑下来，很容易就吃满了。规范帮你设计「渐进式加载」机制——只在需要的时候才读取相关文档，不浪费宝贵的上下文空间。

第二，复杂任务需要「分工」。有些事情脚本做更快（比如调 API 采集数据），有些事情必须让 AI 来判断（比如评估内容质量）。规范帮你明确「谁干什么」，避免用 AI 做机械劳动，也避免让脚本做需要智能的事。

第三，团队协作需要「共识」。当你的 Skill 要给别人用、或者要维护升级的时候，没有规范就是灾难。规范让 Skill 变成「可读的代码」，别人一看就知道怎么用、怎么改。

打个比方

规范之于 Skill，就像建筑规范之于房子。没有规范，你可以盖个能住的房子；有了规范，你盖的房子能通过验收、能卖给别人、能安全地住几十年。

Skill 的本质：给 AI 的「标准作业流程」

❌ 无 Skill：每次重复描述，质量不稳定



✅ 有 Skill：标准化流程，质量稳定一致



★ 核心总结：Skill 是可复用的 AI 工作模板

Skill 的本质：给 AI 的「标准作业流程」

Skill 的骨架：目录结构全解析

一个 Skill 就是一个文件夹。但这个文件夹里要放什么、怎么组织，直接决定了 Skill 好不好用、好不好维护。

最小可用结构：只需要一个文件

最简单的 Skill 长这样：

- **xiangyu-util-format-converter/** — Skill 根目录
 - SKILL.md** — 入口文件，定义 Skill 名称和执行逻辑

没错，只要一个 SKILL.md 文件就能工作。这是「单步骤」Skill，适合简单任务。

标准工作流结构：多步骤 Skill

当任务变复杂，就需要更丰富的结构：

- **xiangyu-social-wechat-creating/** — Skill 根目录
 - default.json
 - python/** — Python 脚本
 - **prompts/** — Agent 提示词模板
 - **presets/** — 预设配置（人格、关键词等）
 - **templates/** — 输出模板
 - step01-init.md
 - step02-collect.md
 - step03-analyze.md
 - step04-output.md

- **SKILL.md** — 入口文件， workflow 概览
- **workflow/** — 步骤执行文档
- **reference/** — 执行时参考资源
- **scripts/** — 可执行脚本
- **config/** — 配置文件
- **credentials/** — API 凭证
- **runs/** — 运行时数据（自动生成）

✖ 结构拆解

把它想象成一个公司的组织架构：

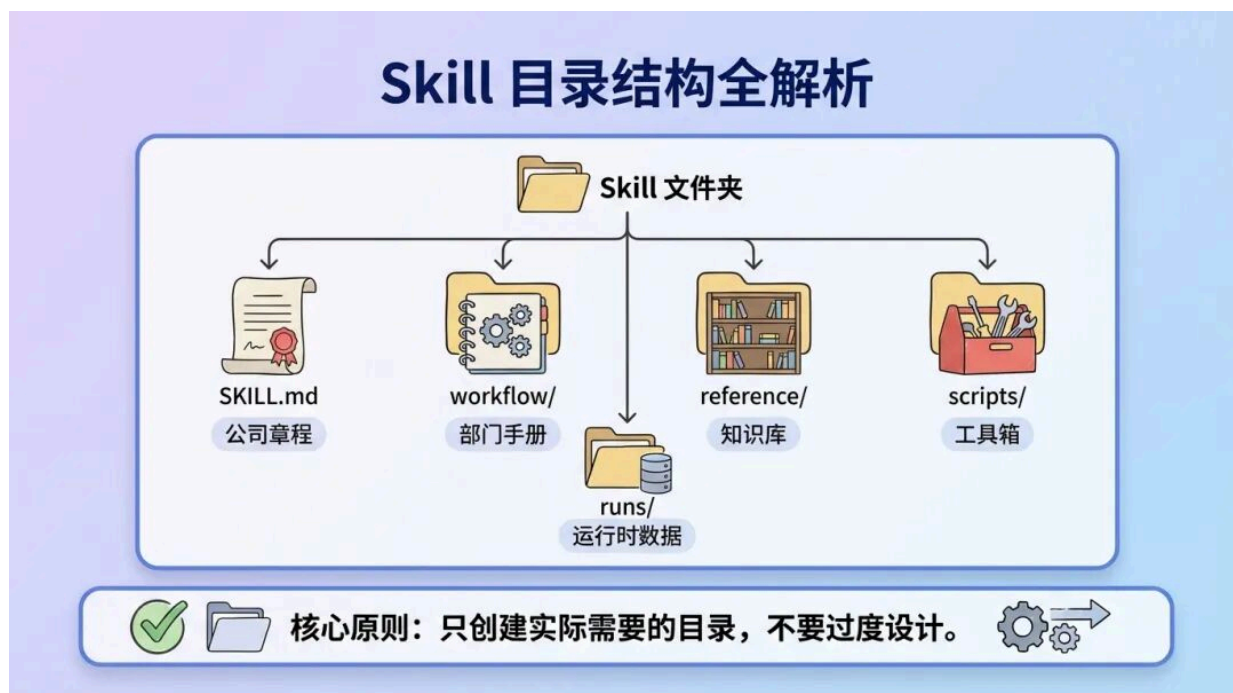
- SKILL.md 是「公司章程」——定义这个 Skill 是干什么的
- workflow/ 是「部门手册」——每个步骤怎么执行
- reference/ 是「知识库」——执行时需要参考的资料
- scripts/ 是「工具箱」——确定性的操作用脚本完成
- runs/ 是「项目档案」——每次运行的数据记录

按需创建：不要过度设计

重要原则：只创建实际需要的目录。

一个简单的格式转换 Skill，不需要 workflow/、reference/、scripts/。直接在 SKILL.md 里写清楚逻辑就行。

规范给你的是「菜单」，不是「套餐」。你根据需要点菜，而不是把菜单上所有菜都点一遍。



Skill 目录结构全解析

目录添加时机：

- **workflow/** — 4+ 步骤，或单步文档超 50 行时添加

- **reference/prompts/** — 有 SubAgent 任务需要提示词模板时添加
- **reference/presets/** — AskUserQuestion 选项需要外部数据时添加
- **scripts/** — 需要调用 API 或处理数据时添加
- **config/** — 需要参数配置时添加
- **credentials/** — 需要 API 凭证时添加

SKILL.md : Skill 的「身份证」

SKILL.md 是 Skill 的入口文件，也是最重要的文件。Claude Code 通过读取这个文件来理解 Skill 是干什么的、什么时候触发、怎么执行。

Frontmatter : 元数据头

每个 SKILL.md 文件开头必须有 YAML frontmatter :

```
---
name: xiangyu-social-wechat-creating
description: 创作公众号长文。当用户说「写长文」「公众号创作」时触发。
---
```

name 字段规则 :

- 最多 64 字符
- 只能用小写字母、数字、连字符
- 不能包含保留词 (anthropic, claude)
- 推荐用 **-ing** 动名词形式

四级命名体系 :

- **第一级 xiangyu-** — 前缀，固定为 xiangyu (或你的命名空间)
- **第二级 {domain}-** — 领域，如 dev / social / content
- **第三级 {target}-** — 对象，如 wechat / github / ppt
- **第四级 {action}** — 动作，如 creating / collecting / syncing

description 字段规则 :

- 最多 1024 字符
- 用第三人称 (不用「我可以帮你」或「你可以用」)
- 格式：**{核心功能}**。**{触发条件}**

 **记住这个**

name 是 Skill 的「身份证号」，全局唯一；description 是「自我介绍」，让 Claude 知道什么时候该用这个 Skill。

触发条件

告诉 Claude 什么关键词会触发这个 Skill :

- 「写长文」「公众号创作」 → 执行完整流程

- 「查看结果」→ 显示最近生成的文章

workflow 定义

这是 SKILL.md 的核心——定义整个工作流的步骤。每个步骤包含以下信息：

- **Step 编号** — 如 01、02、03
- **职责** — 2-6 字，动宾结构（如「数据采集」「内容分析」）
- **执行者** — 谁来干这个活（脚本 / SubAgent / 主 Agent）
- **文档** — 详细执行说明在哪个文件
- **输入** — 这一步需要什么数据
- **输出** — 这一步产出什么数据

🔧 设计洞见

工作流定义是「地图」，让 Claude 和人类都能快速了解整个流程。Claude 不会一开始就读取所有 workflow/*.md 文件——它只看这个定义，然后在执行到某一步时才去读对应的文档。这就是「渐进式加载」的核心。



SKILL.md : Skill 的身份证

执行者：谁来干活？

这是规范中最重要的设计决策之一：每个步骤由谁来执行。

选对执行者，效率翻倍；选错执行者，事倍功半。

四种执行者

1. 脚本 (Script)

适合：确定性操作——输入确定、输出确定、不需要判断

- 调用 API 采集数据
- 文件读写、格式转换
- 数据分批、合并
- 上传下载

优势：零上下文成本，执行快，结果稳定

2. SubAgent

适合：需要 AI 判断的任务——理解、评估、生成、创作

- 内容质量评估
- 文本分析总结
- 创意内容生成
- 异常情况处理

优势：能处理模糊任务，具备理解和判断能力

3. 主 Agent

适合：简单协调任务——读取配置、流程控制

- 解析用户输入
- 读写进度文件
- 协调多个步骤

优势：直接在对话中执行，无需启动新 Agent

4. 准备 SubAgent

适合：大批量数据预处理——需要在执行前先做准备工作

- 统计数据量，计算分批
- 生成批次索引
- 完成后立即压缩上下文

优势：处理海量数据时，避免主 Agent 上下文爆炸

底层原理

为什么要区分这四种执行者？因为它们们的「上下文成本」完全不同：

- 脚本：零成本（只返回一行 JSON 状态）
- SubAgent：中等成本（执行历史会注入主对话）
- 主 Agent：累积成本（所有操作都在主对话中）
- 准备 SubAgent：可控成本（完成后立即压缩）

200k token 的上下文窗口看似很大，但如果不注意控制，几个步骤就能吃满。

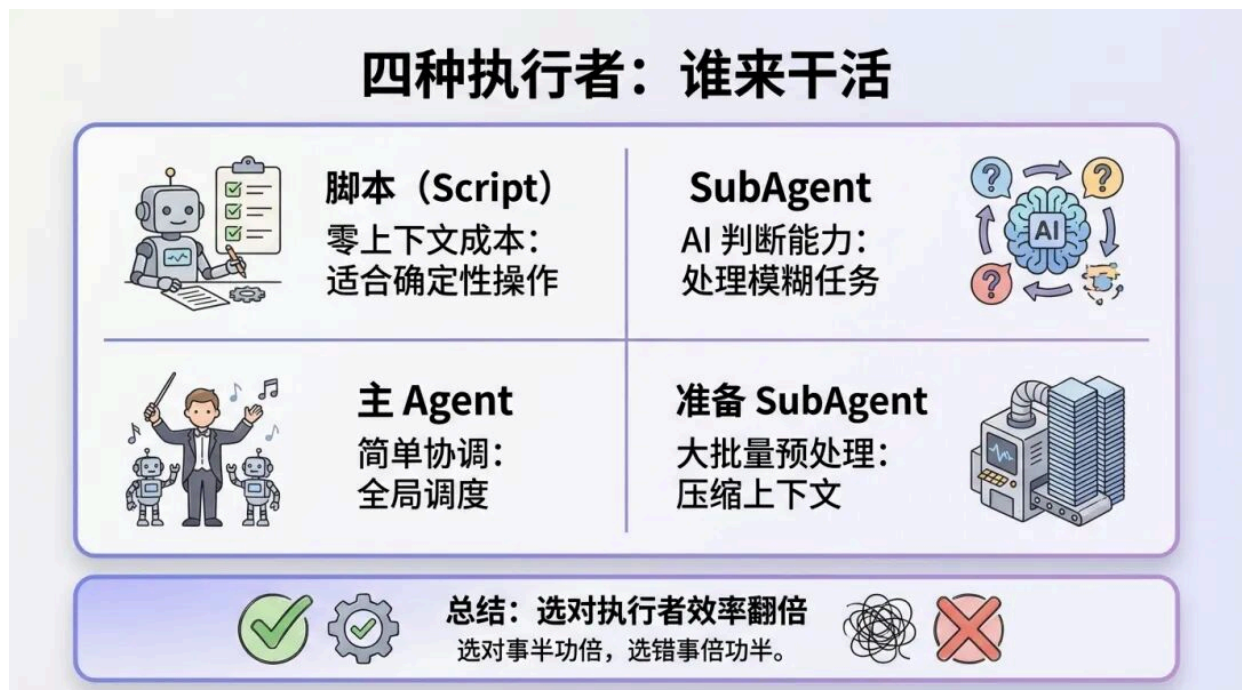
执行者选择决策

问自己：任务需要做什么？

1. 调用 API / 文件操作 / 格式转换？→ 脚本（零上下文成本）
2. 需要 AI 理解 / 判断 / 生成？
 - 数据量 > 500 条？→ 准备 SubAgent（分批）+ SubAgent（执行）
 - 数据量 ≤ 500 条？→ SubAgent
3. 简单协调 / 配置读取？→ 主 Agent

实战案例对比：

- 采集 100 条文章：错误做法是用 SubAgent 调 API，正确做法是用脚本调 API，可节省 99% token
- 评估 30 条内容质量：脚本无法判断，必须用 SubAgent
- 分批 1000 条数据：错误做法是主 Agent 读取全部，正确做法是用准备 SubAgent，可节省 95% token



四种执行者：谁来干活

步骤文档：workflow/*.md 的写法

每个步骤都有一个对应的文档，告诉执行者具体怎么做。

标准模板

步骤文档应包含以下部分：


```
# Step 02: 数据采集 执行者: 脚本
输入: 用户参数
输出: step02-collect/ --- 执行说明: {具体执行步骤描述} 输入文件: 说明每个输入文件的来源和用途,
如 state/config.json 来自 Step 01 输出, 包含用户配置参数 输出文件: 说明每个输出文件的格式和内容,
如 step02-collect/data.json 是 JSON 格式的采集原始数据 脚本执行: cd
{skill_dir}/scripts/python && uv run python collect.py --run-dir {run_dir} 验证检查点:
- 2a: 数据文件存在 - step02-collect/data.json 存在且非空
- 2b: JSON 格式正确 - 可解析为有效 JSON
- 2c: 数据量合理 - items.length > 0 下一步: → Step 03: 内容分析
```

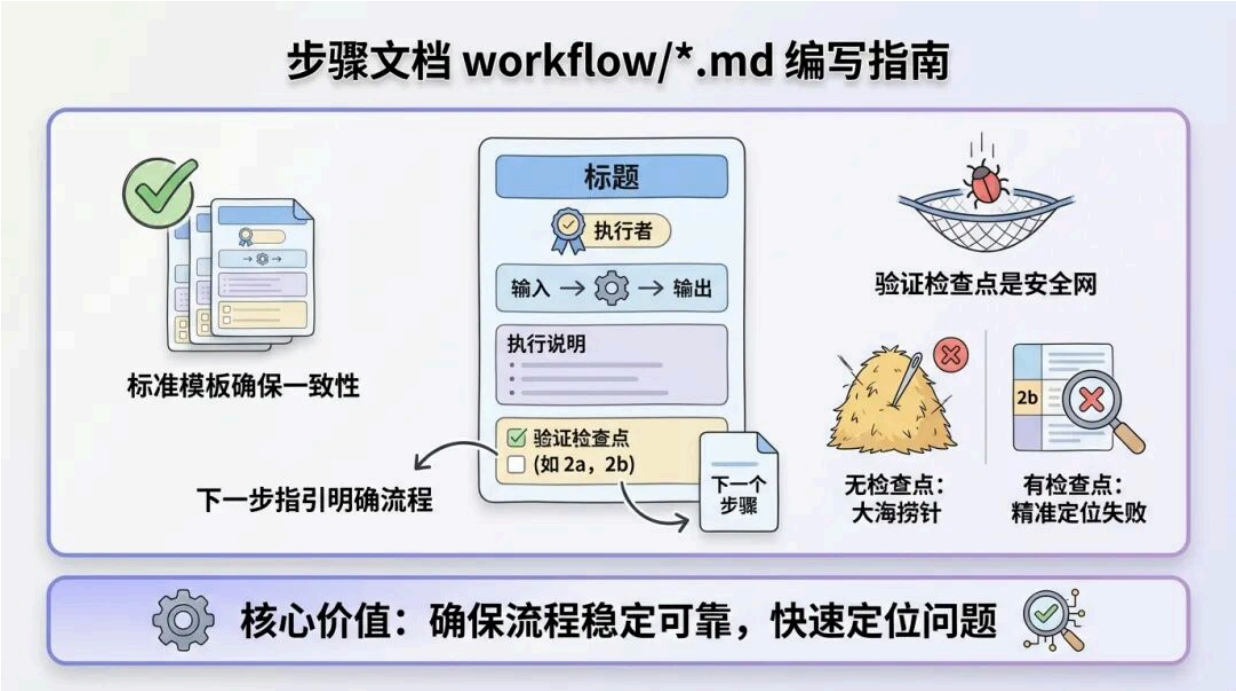
关键要素解析

- 1. 执行说明：用自然语言描述这一步要做什么，让人和 AI 都能看懂。
- 2. 输入/输出文件：明确数据从哪来、到哪去。这是步骤之间的「接口合同」。
- 3. 验证检查点：每个步骤完成后要验证什么。这是质量保障的关键。检查点编号格式：**{步骤号}{字母}**，如 2a、2b、2c。
- 4. 下一步指引：明确告诉执行者接下来去哪。

? 你可能会问

「检查点这么细，会不会太繁琐？」

不会。检查点是你的「安全网」。当 Skill 出问题，你能精确定位到哪个步骤、哪个检查点失败。没有检查点，出了问题就像大海捞针。



步骤文档 workflow/*.md 的写法

运行数据：runs/ 目录的秘密

每次运行 Skill，都会在 runs/ 目录下创建一个新的运行目录，存放本次运行的所有数据。

目录命名规则

格式：`{keyword}-{YYYYMMDD-HHMMSS}`

- **keyword**：从用户输入提取的关键词（如搜索词、用户名）
- **时间戳**：运行开始时间

示例：

- `claude-code-20260130-103000`（调研「Claude Code」）
- `karpathy-20260130-143000`（分析 @karpathy）

目录结构

运行目录包含以下内容：

- **state/** — 【固定】进度状态，存放 progress.json
- **output/** — 【固定】最终输出，存放 result.md 等
- **step02-collect/** — 【动态】步骤输出，根据工作流自动创建
- **step03-analyze/** — 【动态】步骤输出，目录名与步骤文件名对应

progress.json：断点恢复的关键

progress.json 存放运行状态，核心字段包括：

- **keyword** — 运行标识（标准化后）
- **current_step** — 当前步骤
- **completed_batches** — 已完成的批次
- **恢复提示** — `/compact` 后如何恢复

用生活理解技术

progress.json 就像游戏的「存档点」。如果中途退出（或上下文爆满需要压缩），下次进来可以从存档点继续，不用从头开始。

脚本规范：让机器做机器的事

脚本是 Skill 的「执行力」——所有确定性操作都应该用脚本完成。

为什么脚本这么重要？

看一个对比：

用 SubAgent 采集 100 条数据：上下文消耗约 8,000 tokens，执行时间约 30 秒

用脚本采集 100 条数据：上下文消耗约 50 tokens（只返回一行状态），执行时间约 5 秒

节省 99% 的上下文空间。这就是为什么规范强调「能用脚本就不用 Agent」。

Python 脚本核心要点

编写脚本时需遵循以下规范：

1. **返回值规范** — 必须返回 JSON，包含 `ok` 字段表示成功或失败
2. **错误信息** — 失败时返回 `err` 字段，截断到 100 字符以控制输出
3. **输出路径** — 使用 `--run-dir` 参数传入运行目录，不硬编码路径
4. **极简输出** — 只输出状态 JSON，不输出日志和调试信息

调用脚本时，必须 `cd` 到脚本目录确保 `pyproject.toml` 生效。

上下文管理：200k token 的生存法则

这是规范中最容易被忽视、但最容易出问题的部分。

上下文会被什么吃掉？

主要消耗来源：

- **系统 Prompt** — 约 5k tokens
- **SKILL.md** — 约 2-5k tokens
- **步骤文档** — 约 1-3k tokens/步
- **文件读取** — 约 0.1-25k tokens/次
- **SubAgent 返回** — 累积，每个约 10k
- **对话历史** — 累积

关键警告：SubAgent 的执行历史会注入主对话上下文。如果你全并行启动 6 个 SubAgent，就是一次性注入 60k tokens。

分轮执行策略

核心规则：每轮最多启动 2 个 SubAgent，完成后必须 `/compact`。

执行模式：轮次 1 启动 Agent 1 + Agent 2，等待完成后 `/compact`；轮次 2 启动 Agent 3 + Agent 4，等待完成后 `/compact`；以此类推。

为什么是 2 个？ 因为可用上下文空间约 50k tokens（需为 Agent 返回预留），单 Agent 返回约 10k tokens，安全并行度约为 5，保守取值 2 以留有余量。

何时需要 `/compact`

- **准备 SubAgent 完成后** — 强制
- **每轮 SubAgent 完成后** — 推荐
- **整个步骤完成后** — 推荐
- **上下文达到 70%** — 强制

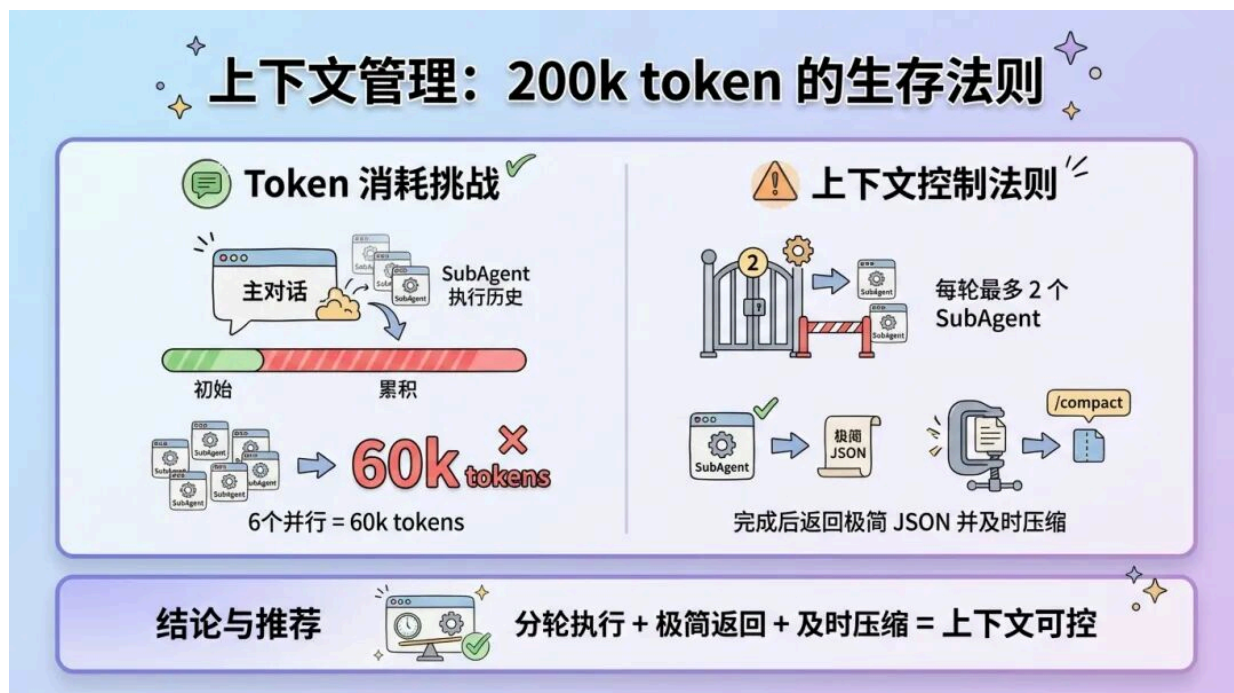
极简返回格式

SubAgent 完成后只返回状态，不返回内容。例如：`{"ok": true, "batch": 1, "count": 30}`

禁止返回：文件内容、分析结果列表、详细日志。

⚡ 三秒版

分轮执行 + 极简返回 + 及时压缩 = 上下文可控。



上下文管理：200k token 的生存法则

AskUserQuestion：优雅地收集用户输入

初始化步骤（通常是 Step 01）需要收集用户输入。规范使用 **AskUserQuestion** 工具来实现。

Claude Code 的限制

- 问题数量 — 1-4 个/次
- 选项数量 — 2-4 个/问题
- header 长度 — ≤12 字符
- Other 选项 — 自动提供，禁止手动添加

四种问题模式

模式 A：核心输入（通常走 Other）

询问关键词、用户名等核心参数，提供几个示例选项，用户通常选 Other 输入自定义值。

模式 B：预设选择

询问目标市场、语言等，从预设文件读取选项供用户选择。

模式 C：模式切换

询问执行模式，如「完整流程」或「仅采集」，固定选项，切换执行路径。

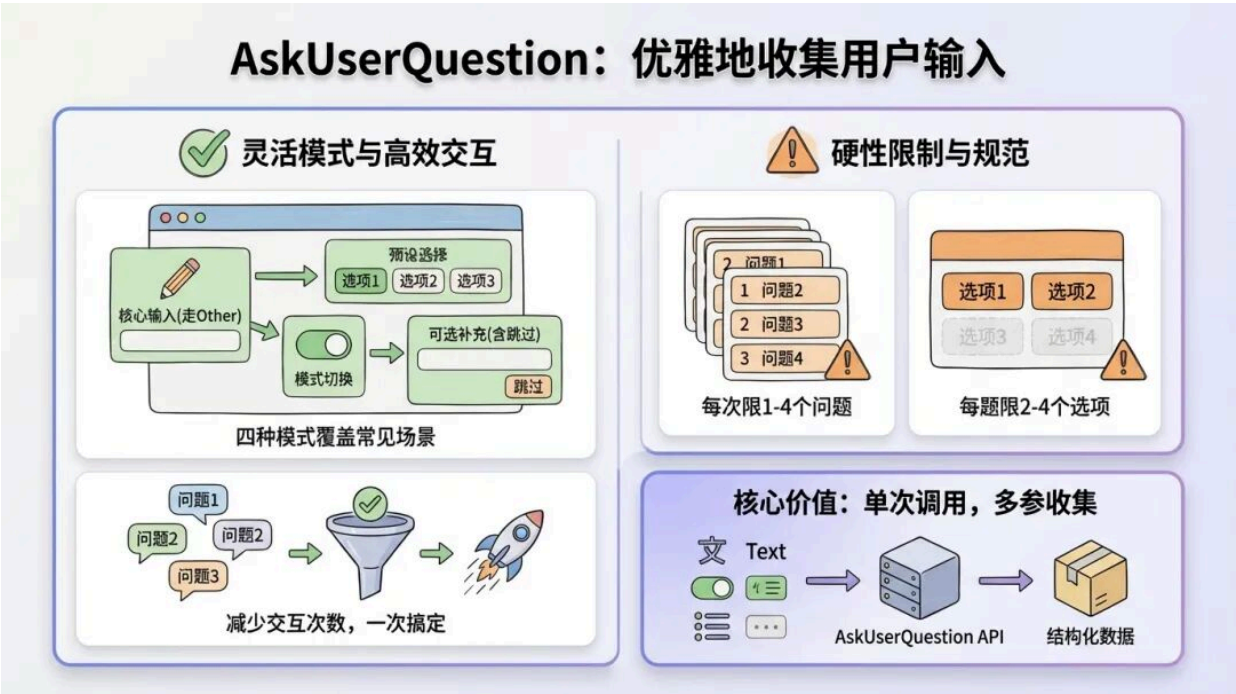
模式 D：可选补充（含跳过）

询问可选参数，如品牌筛选，第一个选项为「跳过」，允许用户不填。

多问题组合策略

单次调用收集多个参数（推荐）：一次 AskUserQuestion 最多包含 4 个问题，同时收集多个参数。

多轮询问（超过 4 个问题）：第一轮收集核心参数，第二轮根据第一轮结果收集补充参数。



AskUserQuestion：优雅地收集用户输入

错误处理与恢复：让 Skill 更健壮

好的 Skill 要能跑，更要能从错误中恢复。

五层验证模式

- 1. 文件存在且非空 — 失败则重试
- 2. 格式正确 (JSON/MD 可解析) — 失败则重试
- 3. 字段完整 (必要字段存在) — 失败则重试
- 4. 值范围有效 (如 $0 \leq \text{score} \leq 100$) — 标记异常
- 5. 业务逻辑 (满足业务规则) — 标记失败

错误分类与处理

- 临时错误 (网络超时、API 限流) — 可重试，指数退避重试
- 永久错误 (参数错误、格式错误) — 不可重试，终止并报告
- 权限错误 (401/403、Token 过期) — 不可重试，终止并提示检查凭证
- 资源错误 (上下文溢出、磁盘满) — 不可重试，清理后重试

恢复流程

当 `/compact` 或中断后恢复：

1. 读取 `state/progress.json`
2. 查看 `恢复提示` 字段
3. 确认执行器类型（是用 Task 还是直接执行）
4. 从断点继续

多模式 Skill：一个 Skill 多种玩法

有时候一个 Skill 需要支持多种执行模式。比如公众号克隆 Skill，可以从博主主页采集（Clone 模式），也可以从已有数据分析（Timeline 模式）。

目录组织

多模式 Skill 的 workflow/ 目录按模式分子目录：

- `workflow/clone/` — Clone 模式的步骤文档
- `workflow/timeline/` — Timeline 模式的步骤文档
- `workflow/shared/` — 共享步骤文档

触发词设计

不同关键词触发不同模式：

- 「克隆」「`clone`」→ Clone 模式，从博主主页采集
- 「时间线」「`timeline`」→ Timeline 模式，从已有数据分析

运行目录命名

多模式 Skill 的运行目录带模式前缀：

- `clone-karpathy-20260130-103000`
- `timeline-sama-20260130-143000`

多模式 vs 多 Skill

- 选择多模式 — 输出格式相同 + 共享大量逻辑
- 选择多 Skill — 输出格式不同 + 逻辑差异大

平台约束：Claude Code 的硬性限制

最后，必须了解 Claude Code 的硬性限制，这些是规范设计的「物理边界」。

上下文窗口

Claude Sonnet/Haiku/Opus 模型上下文窗口为 200k tokens，有效可用约 180k。

工具限制

- **Read** — 25k tokens 上限，2000 行截断
- **Edit** — 必须先 Read
- **Write** — 已存在文件必须先 Read
- **Bash** — 30k 字符输出截断，默认 2 分钟超时
- **MCP** — 25k tokens 输出上限
- **Task** — 建议每轮 ≤ 2 个并行
- **Skill** — 15k 字符预算

Skill 安装路径

- 用户级 (默认) — `~/.claude/skills/<name>/`
- 项目级 — `.claude/skills/<name>/`

一键复刻

复制下面这段提示词给 Claude Code，从零创建你的第一个 Skill：

你是 AI 工作流架构师。请帮我创建一个完整的 Skill，遵循翔宇 Skill 规范。 ****Skill 信息****：

- 名称：xiangyu-`{domain}`-`{target}`-`{action}` (请根据我的需求填写)
 - 功能：{我会告诉你具体功能} ****创建要求****： 1. ****目录结构****：
 - SKILL.md：入口文件，包含 frontmatter (name + description)、触发条件、工作流定义
 - workflow/：步骤文档 (stepNN-action.md 格式)
 - 按需创建 reference/、scripts/、config/ 2. ****SKILL.md 规范****：
 - frontmatter 用 kebab-case
 - name 最多 64 字符，小写+数字+连字符
 - description 第三人称，格式「{功能}。{触发条件}」
 - 工作流定义包含：Step / 职责 / 执行者 / 文档 / 输入 / 输出 3. ****步骤文档规范****：
 - 文件名：stepNN-`{action}`.md (NN 两位数字)
 - 包含：执行者、输入、输出、执行说明、验证检查点 4. ****执行者选择****：
 - 确定性操作 (API 调用、文件处理) → 脚本
 - 需要 AI 判断 (评估、生成) → SubAgent
 - 简单协调 → 主 Agent 5. ****上下文管理****：
 - SubAgent 每轮最多 2 个
 - 完成后返回极简 JSON (`{"ok": true, ...}`) 6. ****输出目录****：
 - 固定目录：state/、output/
 - 动态目录：step`{NN}`-`{action}`/ (与 workflow 文件名对应)
- 请问你想创建什么功能的 Skill？告诉我具体需求，我会按照规范帮你生成完整的文件结构和内容。

一键复刻：从零搭建你的第一个 Skill



一键复刻：从零搭建你的第一个 Skill

参考资料

翔宇 Skill 规范知识库